

CS3810 Mini-Project 1 — Vacuum World Search Algorithms

Hannah Dees and Alexis Stacy

Introduction to Artificial Intelligence CS3810

Dr. Wang

2 October 2026

Introduction

This project focuses on implementing three search algorithms, DFS, A*, and IDA, in the Vacuum World environment. We evaluated the performance of these algorithms using multiple heuristics (h0, h1, h2) and tested them across several grid configurations. The evaluation measured cost, nodes expanded, frontier size, iterations, and runtime. Our analysis centers on how different heuristics can affect the efficiency of each search algorithm.

Design Decisions

For the implementation, we represented each state as the robot's position along with a frozenset containing the dirty cells that are remaining. We used a tuple for the robot's position and a frozenset for the dirty cells because the states need to be stored in sets and dictionaries. In the result () method, we created new states instead of changing existing ones to prevent accidental changes. For DFS, we used a deque as a stack instead of using recursion. We also used an explored set so the search would not keep going in circles. We reversed the actions when adding them to the stack so they would follow the order given in the assignment. For A*, we used a priority with $g + h$ to decide which state to check next and used a dictionary to keep track of the lowest cost we found each time. For IDA*, we used the current path to detect cycles rather than keeping a permanent explored set to use less memory. We mostly followed the required design and made choices based on what worked best with the algorithms.

Heuristic Analysis

H1 Analysis

H1 returns the number of dirty cells remaining in each state. Each dirty cell requires at least one CLEAN action, and CLEAN costs 1. If the robot were already standing on a dirty cell, it would still need to spend one unit of cost to clean the cell. Therefore, the true remaining cost can never be less than the number of dirty cells. H1 never overestimates the optimal cost, so it is admissible.

H2 Analysis

H2 returns the number of dirty cells plus the Manhattan distance to the nearest dirty cell. The robot must move to at least one dirty cell before cleaning anything, and Manhattan distance is a lower bound on the number of moves required. Obstacles can force detours but never reduce the distance. Since h2 adds two independent lower bounds (movement + cleaning), it never overestimates the true remaining cost and is admissible.

H3 Analysis

H3 was not implemented; results are marked as SKIP.

Experimental Results

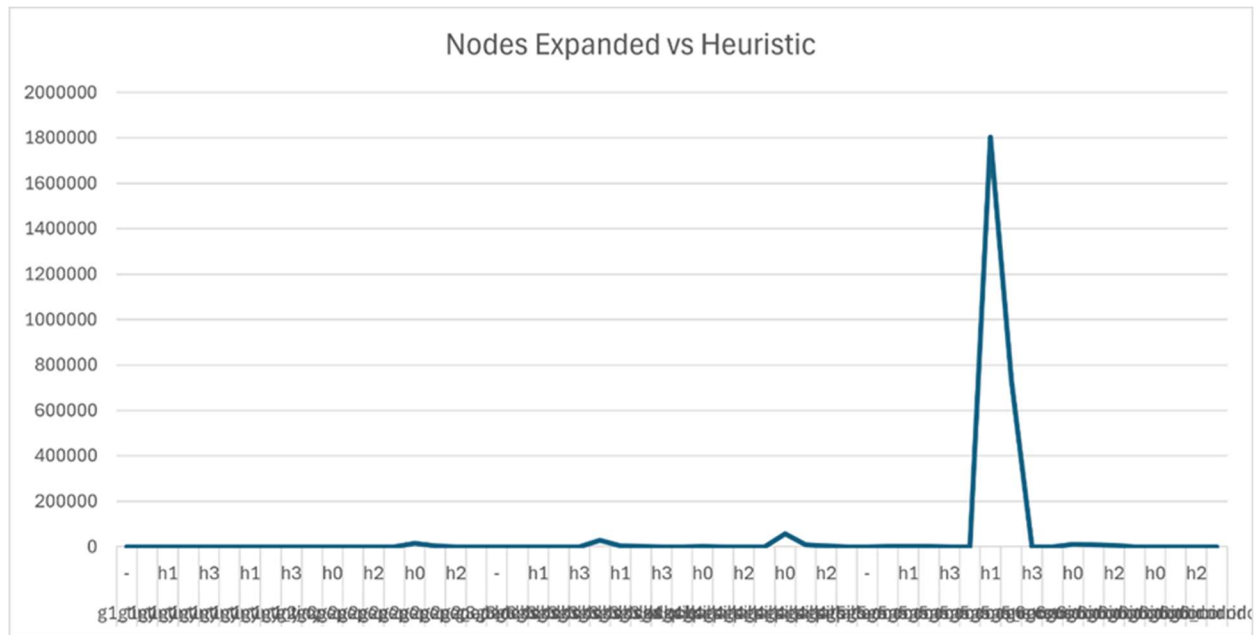
Table

Grid	Algorithm	Heuristic	Cost	Nodes Expanded	Max Frontier	Iterations	Seconds
g1_tiny	DFS	—	12	16	5	—	0.0001

g1_tiny	A*	h0	7	21	4	—	0.0002
g1_tiny	A*	h1	7	20	5	—	0.0002
g1_tiny	A*	h2	7	13	4	—	0.0001
g1_tiny	A*	h3	SKIP	—	—	—	—
g1_tiny	IDA*	h0	7	95	8	1	0.0005
g1_tiny	IDA*	h1	7	52	6	1	0.0002
g1_tiny	IDA*	h2	7	27	4	1	0.0002
g1_tiny	IDA*	h3	SKIP	—	—	—	—
g2_open	DFS	—	36	48	32	—	0.0002
g2_open	A*	h0	12	98	17	—	0.0005
g2_open	A*	h1	12	96	18	—	0.0005
g2_open	A*	h2	12	76	21	—	0.0005
g2_open	A*	h3	SKIP	—	—	—	—
g2_open	IDA*	h0	12	15586	13	1	0.0711
g2_open	IDA*	h1	12	3821	10	1	0.0159
g2_open	IDA*	h2	12	616	4	1	0.0039
g2_open	IDA*	h3	SKIP	—	—	—	—

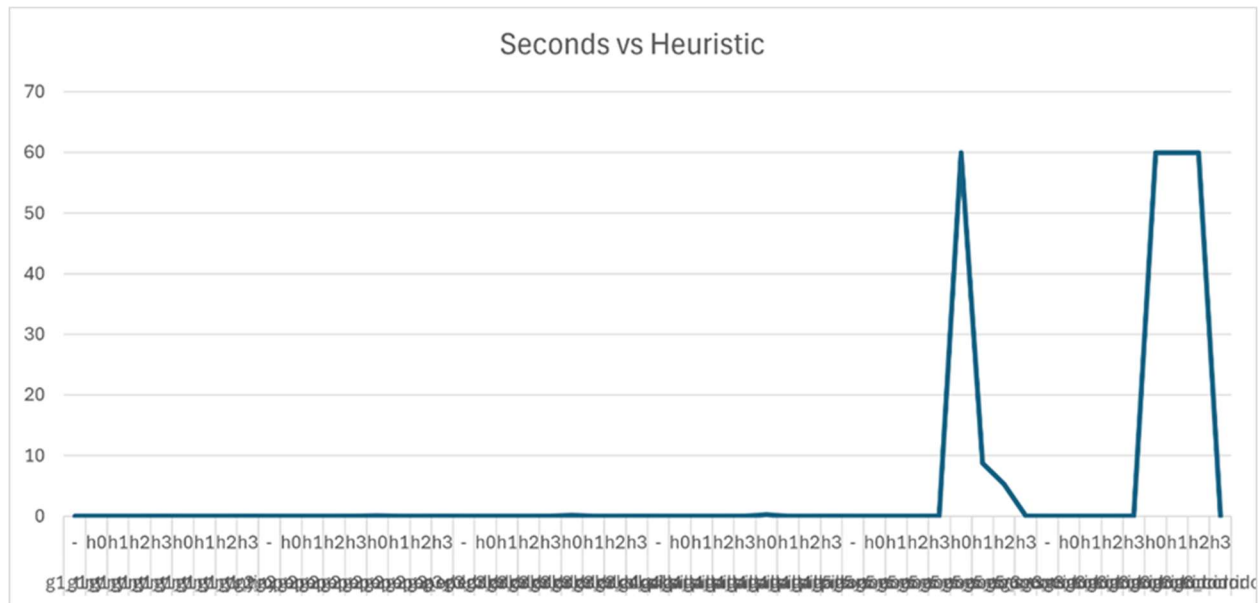
g3_blocks	DFS	—	43	69	34	—	0.0003
g3_blocks	A*	h0	15	411	11	—	0.0013
g3_blocks	A*	h1	15	331	71	—	0.0018
g3_blocks	A*	h2	15	256	87	—	0.0015

Plot One: Nodes Expanded Vs Heuristic



Nodes expanded for each heuristic across all grids.

Plot Two: Seconds Vs Heuristic (Runtime)



Runtime for each heuristic across all grids.

Discussion

Question One

DFS expands far fewer nodes because it commits early to a single path as deep as possible before evaluating alternatives. This means it does not explore the other branching factors at each of the other levels; hence, the node count stays small. DFS also returns much worse solutions because there is no cost awareness. It does not consider path length or dirt locations. IF the first deep path is long or loops around the grid, the DFS will still return it. Overall, DFS only explores a few nodes and ignores most of the search space. DFS also ignores cost and goals, then gives out bad solutions.

Question Two

A* with H0 (zero heuristic) expands a huge number of nodes because there is no guidance or limitations. A* with H1(the number of dirty cells) expands to fewer nodes because it

recognizes the dirty cells as more computational effort. A* with H2 (distance to nearest dirt) expands even less nodes because it provides a more specific solution.

Question Three

IDA* expands more nodes than A* because it repeatedly performs depth limited searches, restarting from the root each time. When restarting from the root, the cost bound increases. This wastes work that A* avoids by storing nodes in memory. However, IDA* uses very little memory, making it ideal if there is limited memory space. IDA* trades time for memory, so if memory is not a focus or need, IDA* would be the method to pick.

Question Four

A state is defined by the robot's position and the subset of dirty cells remaining. The state space "blows up" as dirty cells increase, with a $m \times n$ grid and k dirty cells, the robot can be in any of mn positions and any subset of dirt may remain, producing $mn \cdot 2^k$ states, and this exponential factor of 2^k is why search quickly becomes complex. Every additional dirty cell doubles the amount of possible dirt remaining, and the search must consider all of them.

Question Five

The hardest grid is the one with many dirty cells, dirt far apart, obstacles, or long detours; these structures increase the number of distinct dirt-remaining subsets the search must consider. A larger grid scattered with dirt would be a combination of explosions because it maximizes both path length and dirty cells.

The Roomba Question

Assumption: Perfect knowledge of dirt locations. If the robot did not know where the dirt was in advance, the entire implementation would break because the search problem would no longer be a deterministic shortest path problem. The robot would have to explore the entire grid to discover all the dirty cells, meaning the state space would include beliefs about where the cells may be. This then increases cost and time. The problem becomes fundamentally harder because the robot must plan under uncertainty rather than simply compute an optimal path on a fully known grid.

Reflection and Overall Analysis

The experiments demonstrate that heuristics dramatically affect search performance in the Mini Vacuum World. We found that H0 provides no guidance, H1 offers a basic lower bound, and H2 significantly improves efficiency by guiding the search more intelligently. A* with H2 consistently yielded the most effective results, finding optimal solutions with minimal node expansion and reduced computational effort. Although IDA* also benefits from heuristics, its reliance on repeated iterations to deepen the search contributes to slower performance compared to A*. When running tests using `python experiments.py` in the terminal, at first, it timed out when it got to g5. Then, when running `python run_tests.py` in the terminal, it did complete all the way to g6, without a TIMEOUT. Rows were written to `results.csv` after every test. These findings emphasize the critical value of carefully designed, admissible heuristics in informed search algorithms, not only in the Mini Vacuum World, but potentially in a wide range of complex problem-solving scenarios. This project enhanced our understanding of how theoretical properties of heuristics translate directly into practical performance gains and emphasized the importance of selecting and designing heuristics tailored to the specific characteristics of the problem domain.

